

Міністерство освіти і науки України
Національний університет “Львівська політехніка”
Кафедра систем автоматизованого проектування
Інститут комп’ютерних наук та інформаційних технологій



Звіт
про виконання лабораторної роботи № 1
з дисципліни: “Штучні нейронні мережі”
На тему: “Нейромережі прямого поширення”.

Виконав студент групи ШТНМ-САП-101

Ткач Д. О.

Прийняв:

Соколовський Я. І.

Львів 2025

Мета роботи: вивчити архітектури формального нейрона, багатошарового персептрона, а також методи їхнього навчання; ознайомитися з програмними продуктами, що моделюють персептрони.

Завдання до роботи:

1. Використовуючи документацію програмних засобів (Python з бібліотекою Keras або Matlab, або Statistica Neural Networks, або інше), вивчити їх архітектуру і компоненти, призначені для моделювання і навчання нейромереж прямого поширення, введення і виведення даних, підготовки звітів (виведення і відображення результатів роботи).

2. Сформуванати навчальну і тестову вибірки даних. Визначити за табл. 1 кількості шарів та нейронів у шарах для побудови багатошарових персептронів, а також функцію активації для завдання п. 4.

3. Використовуючи програмний засіб для відповідних варіанту студента вхідних даних вирішити прикладну задачу на основі одношарового та багатошарового персептронів, використовуючи різні методи навчання та різні функції активації.

4. Змінюючи значення кроку навчання, для одношарового та багатошарового персептронів з заданою функцією активації, що відповідає номеру варіанту студента (див. табл. 1) дослідити, як впливає величина кроку навчання на час навчання. Побудувати графіки залежності часу навчання персептронів від величини кроку навчання.

5. Зберегти у файлі початкові параметри нейромоделей, результати їхнього навчання (матрицю ваг та структуру з зазначенням функцій активації і кількості нейроелементів у шарах) та роботи для навчальної та контрольної вибірок (помилку класифікації (оцінювання), значення на входах і виході, час навчання, час класифікації).

6. Результати виконання пп. 3–5 занести у таблицю, стовпці якої повинні мати назви: назва архітектури нейромережі, кількість шарів, кількість нейронів у шарах, функції активації нейронів у шарах, метод навчання, час навчання, час класифікації (оцінювання) навченої мережі для навчальної вибірки, помилка класифікації (оцінювання) для навчальної вибірки, час класифікації (оцінювання) навченої мережі для тестової вибірки, помилка класифікації (оцінювання) для тестової вибірки.

7. Проаналізувати отримані результати і зробити висновки про те, як впливає вид функції активації формального нейрона на час навчання і час класифікації (оцінювання), а також величину помилки навчання і

класифікації (оцінювання) одношарового персептрона; як впливають вид функції активації і вид коригувального правила ваг (метод навчання) на час навчання і класифікації (оцінювання), а також величину помилки навчання та класифікації багатшарового персептрона; яка модель нейромереж прямого поширення краще підходить для вирішуваної задачі.

Індивідуальне завдання:

Номер варіанта	Багатшаровий персептрон		Назва функції активації
	Кількість шарів	Кількості нейронів у шарах	
18	4	7–3–4–1	тангенційна сигмоїдна

Короткі теоретичні відомості:

Штучна нейронна мережа прямого поширення – це багатшарова нейронна мережа без зворотних зв'язків, у якій вихід кожного нейрона шару з'єднаний із входами всіх нейронів наступного шару, а для перетворення сигналу вхідного збудження кожного нейрона прихованого і вихідного шару у вихідний сигнал використовується нелінійна активаційна функція. Поріг спрацьовування кожного нейрона реалізується за допомогою bias-нейрона, вихід якого завжди має значення 1, а ваговий коефіцієнт зв'язку цього нейрона з іншими нейронами мережі налаштовується в процесі навчання (рис. 1).

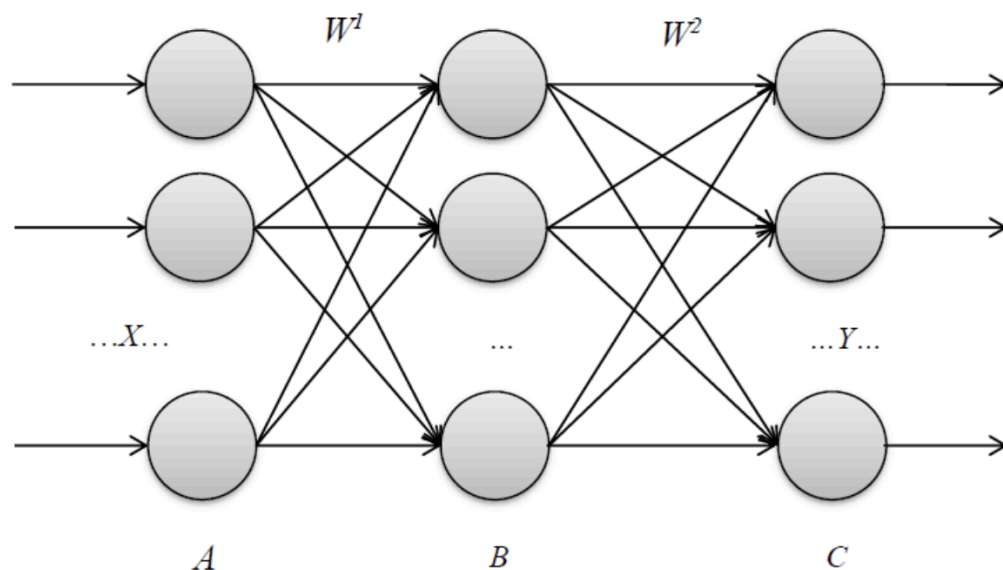


Рис. 1. – Приклад двошарової штучної нейронної мережі

Шар А містить L нейронів, шар В – M нейронів, шар С – N нейронів. Нейрони шару А ніяких функцій не виконують, окрім розподілу сигналів. Розмірність вхідного шару А мережі прямого поширення відповідає розмірності вектора вхідних даних задачі (табл. 1.)

Таблиця 1. – Початкові дані.

X_1	X_2	...	X_L	D_1	D_2	...	D_N
x_{11}	x_{12}	...	x_{1L}	d_{11}	d_{12}	...	d_{1N}
x_{21}	x_{22}	...	x_{2L}	d_{21}	d_{22}	...	d_{2N}
...	...	...	...	...	...	...	...
x_{k1}	x_{k2}	...	x_{kL}	d_{k1}	d_{k2}	...	d_{kN}

Мережі прямого поширення можуть функціонувати в трьох режимах: навчання, тестування і погін. Для навчання двошарових штучних нейронних мереж прямого поширення традиційно використовуються різні варіанти методу зворотного поширення похибки. Термін «зворотне поширення похибки» (back propagation) означає: – ефективний метод обчислення похідних; – алгоритм оптимізації з використанням цих похідних, що дозволяє налаштовувати вагові коефіцієнти з метою мінімізації помилки. Алгоритм зворотного поширення помилки реалізує градієнтний метод мінімізації опуклого (звичайного квадратичного функціонала) помилки в багатошарових мережах прямого поширення, що використовують моделі нейронів з диференціальними функціями активації.

Хід роботи:

Для виконання даного завдання мною було використано програмні засоби Python та бібліотеку Keras, вивчено їх архітектуру і компоненти, призначені для моделювання і навчання нейромереж прямого поширення, введення і виведення даних, підготовки звітів (виведення і відображення результатів роботи).

Наступним кроком використовуючи програмний засіб для відповідного варіанту вхідних даних вирішую прикладну задачу на основі одношарового та багатошарового перцептронів.

Першим кроком у програмі генерую дані для перцептрона:

```
# Генерування даних
X = np.random.rand(1000, 7)
y = np.random.rand(1000, 1)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

В моєму випадку було згенеровано 1000 записів з відповідною кількістю ознак.

Далі, описую функцію роботи багат шарового перцептрона:

```
# Багат шаровий перцептрон (MLP)
def create_mlp_model(learning_rate):
    model = Sequential([
        Dense(3, input_shape=(7,), activation='tanh'),
        Dense(4, activation='tanh'),
        Dense(1, activation='tanh')
    ])
    optimizer = SGD(learning_rate=learning_rate)
    model.compile(optimizer=optimizer, loss='mse', metrics=['mae'])
    return model
```

Використовую бібліотеку Keras, яка допомагає у створенні перцептрона.

Далі, створив метод для одно шарового перцептрона:

```
# Одно шаровий перцептрон (SMP)
def create_single_layer_perceptron(learning_rate):
    model = Sequential([
        Dense(1, input_shape=(7,), activation='tanh')
    ])
    optimizer = SGD(learning_rate=learning_rate)
    model.compile(optimizer=optimizer, loss='mse', metrics=['mae'])
    return model
```

Задаю бажані кроки для навчання та ініціалізую масиви для збереження результатів:

```
# Крок навчання
learning_rates = [0.001, 0.01, 0.05, 0.1]
mlp_results = []
smp_results = []
```

Далі, для кожного кроку навчання запускаю створення перцептрону та зберігаю результати:

```
for lr in learning_rates:
    model = create_mlp_model(lr)

    start_time = time.time()
    model.fit(X_train, y_train, epochs=50, verbose=0)
    training_time = time.time() - start_time

    start_eval = time.time()
    train_loss, _ = model.evaluate(X_train, y_train, verbose=0)
```

```

train_eval_time = time.time() - start_eval

start_eval = time.time()
test_loss, _ = model.evaluate(X_test, y_test, verbose=0)
test_eval_time = time.time() - start_eval

mlp_results.append({
    'Тип': 'MLP',
    'Крок навчання': lr,
    'Час навчання': training_time,
    'Час класифікації (train)': train_eval_time,
    'Помилка (train)': train_loss,
    'Час класифікації (test)': test_eval_time,
    'Помилка (test)': test_loss
})

```

Використовуючи бібліотеку `matplotlib`, виводжу результати у вигляду графіку для кожного з перцептронів:

```

# Графік для MLP
plt.figure(figsize=(8, 5))
plt.plot(df_mlp['Крок навчання'], df_mlp['Час навчання'], marker='o')
plt.title('MLP: Залежність часу навчання від learning rate')
plt.xlabel('Крок навчання')
plt.ylabel('Час навчання (сек)')
plt.grid(True)
plt.show()

```

Також, всі отримані дані зберігаю у вигляді таблиці.

Після запуску програми, отримую графіки залежності кроку навчання на час навчання для двох видів перцептронів:

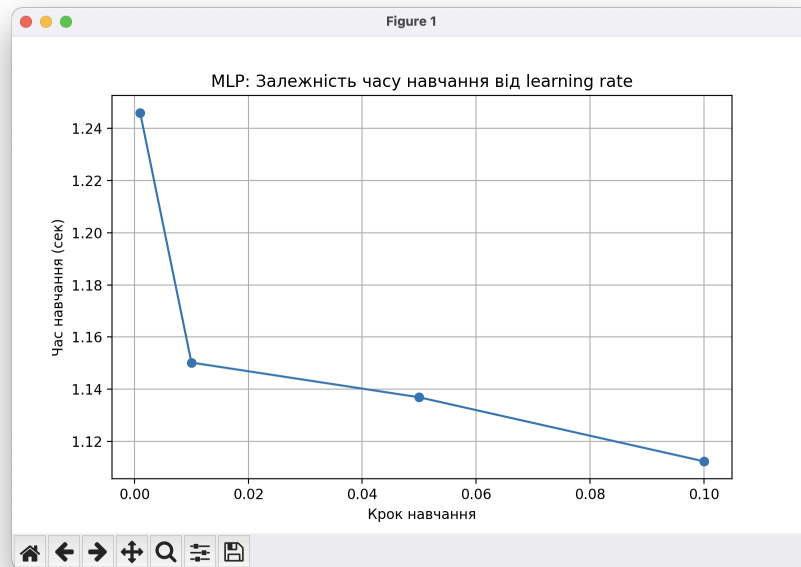


Рис. 1. Графік залежності для багатошарового перцептрону

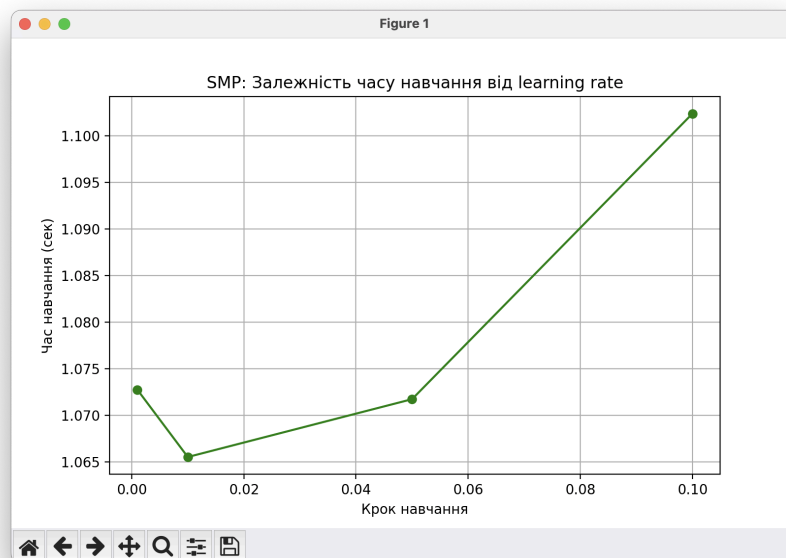


Рис. 2. Графік залежності для одношарового перцептрону

Програма збирає згенеровані дані у файл:

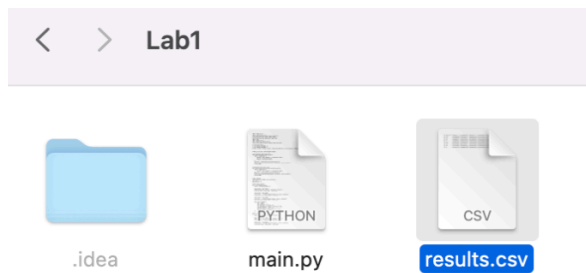


Рис. 3. Згенерований файл

Тип	Крок навчання	Час навчання	Час класифікації (train)
MLP	0.001	1.2459499835968018	0.06211996078491211
MLP	0.01	1.1502180099487305	0.0619351863861084
MLP	0.05	1.136909008026123	0.06708621978759766
MLP	0.1	1.1123650074005127	0.06428289413452148
SMP	0.001	1.0727438926696777	0.05636405944824219
SMP	0.01	1.0655181407928467	0.056610107421875
SMP	0.05	1.0717239379882812	0.05762600898742676
SMP	0.1	1.1023590564727783	0.056938886642456055

Помилка (train)	Час класифікації (test)	Помилка (test)
0.08840558677911758	0.0632791519165039	0.08460965007543564
0.08461690694093704	0.06156110763549805	0.08971317112445831
0.08317065238952637	0.06128692626953125	0.08452365547418594
0.08250346779823303	0.06214284896850586	0.08524298667907715
0.13054631650447845	0.05618596076965332	0.12727074325084686
0.09147637337446213	0.05679583549499512	0.09350047260522842
0.08202090114355087	0.12860679626464844	0.08514530211687088
0.08249887824058533	0.05615687370300293	0.08556570112705231

Табл. 1. Параметри нейромоделі

У ході виконання лабораторної роботи були проведені дослідження з використанням нейронних мереж прямого поширення — одношарового (SMP) та багатшарового (MLP) перцептронів. Основною метою експериментів було визначити, як різні функції активації та методи навчання впливають на ефективність роботи моделей. Аналіз охоплював порівняння часу навчання, часу класифікації та точності для обох типів мереж.

Для одношарового перцептрона було обрано функцію активації $\tanh$. Як показали експериментальні дані одношаровий перцептрон по своїй точності відстає від багатшарового перцептрона, особливо на складніших вхідних даних. Основна причина полягає в тому, що одношаровий перцептрон за своєю природою здатен моделювати лише лінійні залежності між вхідними та вихідними змінними. У випадках, де зв'язок є складнішим, така модель демонструє вищу помилку як на навчальних, так і на тестових вибірках. Водночас час навчання та класифікації для SMP був найнижчим, що логічно, адже обчислення відбуваються через один шар без додаткових прихованих шарів.

Багатшаровий перцептрон продемонстрував кращі результати щодо точності. Завдяки наявності кількох прихованих шарів і використанню тієї ж функції активації $\tanh$, модель змогла ефективно моделювати нелінійні залежності та досягти меншого рівня помилки. Проте це супроводжувалося більшим часом навчання, що також було очікуваним, адже кількість параметрів, які потрібно було оптимізувати, зросла.

Також було досліджено, як змінюється поведінка моделей при варіації **кроку навчання**. Для обох типів перцептронів спостерігалася схожа картина: надто малий крок уповільнює збіжність моделі, а надто великий — може призводити до нестабільного навчання або навіть до дивергентної поведінки. Найкращі результати досягаються при помірних значеннях кроку, що забезпечували баланс між швидкістю навчання та стабільністю.

Загалом, багатшарові нейронні мережі виявилися більш придатними для вирішення поставленої задачі. Вони забезпечують вищу гнучкість, здатність до апроксимації складних функцій і дають кращі результати за точністю, хоча і вимагають більше ресурсів та часу на

навчання. Одношарові мережі можуть бути корисні лише для найпростіших задач або як базові моделі для порівняння.

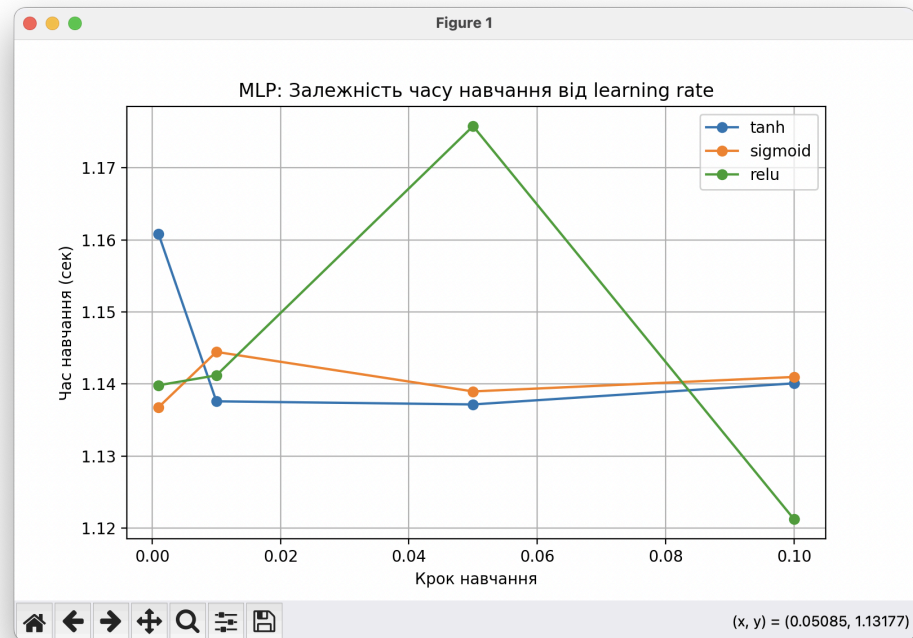


Рис. 4. Порівняння декількох функцій активації на багатошаровому перцептроні

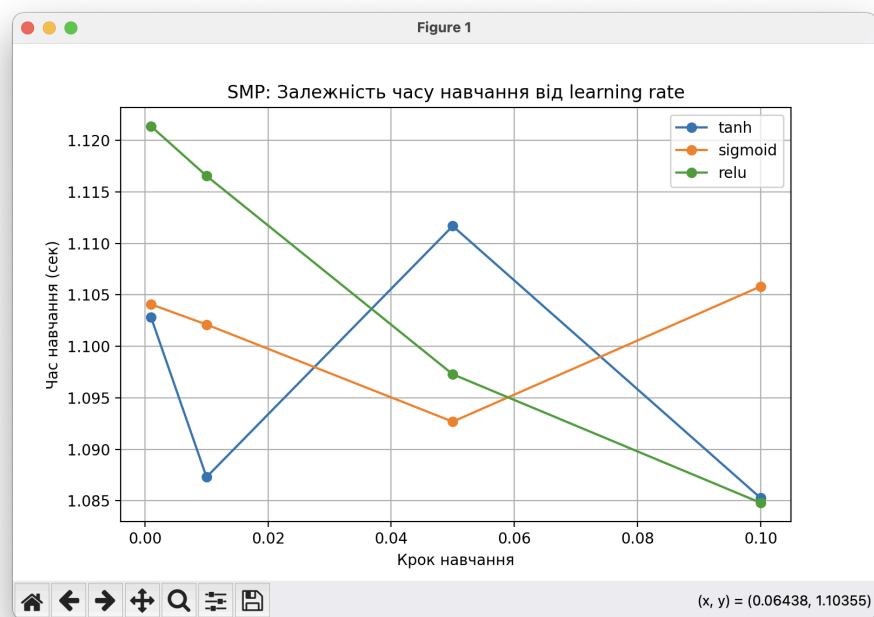


Рис. 5. Порівняння декількох функцій активації на одношаровому перцептрі

Модел	Активация	Крок навчання	Час навчання	Помилка (train)	Помилка (test)
MLP	tanh	0.001	1.1607921123504639	0.08736027032136917	0.09300320595502853
SMP	tanh	0.001	1.1027979850769043	0.15363699197769165	0.1717955768108368
MLP	tanh	0.01	1.1375961303710938	0.08673252910375595	0.097235307097435
SMP	tanh	0.01	1.08729887008667	0.09537939727306366	0.11565417051315308
MLP	tanh	0.05	1.1371629238128662	0.0832018256187439	0.09386995434761047
SMP	tanh	0.05	1.1116771697998047	0.08266064524650574	0.09715835750102997
MLP	tanh	0.1	1.1400890350341797	0.08282157778739929	0.09652698785066605
SMP	tanh	0.1	1.0852739810943604	0.08255447447299957	0.09675522893667221
MLP	sigmoid	0.001	1.1367847919464111	0.09534359723329544	0.1019693911075592
SMP	sigmoid	0.001	1.104071855545044	0.09481322020292282	0.10847309976816177
MLP	sigmoid	0.01	1.1444520950317383	0.08340815454721451	0.09319624304771423
SMP	sigmoid	0.01	1.102113962173462	0.09093744307756424	0.09539549052715302
MLP	sigmoid	0.05	1.1389679908752441	0.08348910510540009	0.09267579764127731
SMP	sigmoid	0.05	1.0926809310913086	0.08473508059978485	0.09899118542671204
MLP	sigmoid	0.1	1.1409759521484375	0.08361972868442535	0.0922621563076973
SMP	sigmoid	0.1	1.10580706596	0.08299365639	0.096046954

			37451	686584	39338684
MLP	relu	0.001	1.13982391357 42188	0.09341373294 591904	0.108022488 65365982
SMP	relu	0.001	1.12136077880 85938	0.21899677813 05313	0.239316925 406456
MLP	relu	0.01	1.14120006561 2793	0.08375332504 51088	0.095798224 21073914
SMP	relu	0.01	1.11654710769 65332	0.09453681856 393814	0.103241890 66886902
MLP	relu	0.05	1.17578291893 00537	0.32270646095 27588	0.341869443 65501404
SMP	relu	0.05	1.09728288650 5127	0.08272013813 257217	0.097069628 5367012
MLP	relu	0.1	1.12125182151 79443	0.08311776816 84494	0.095405414 70050812
SMP	relu	0.1	1.08480882644 65332	0.08437255024 909973	0.099112123 2509613

Табл. 2. Параметри нейромоделі для різних функцій активації

Вплив функції активації на результати можна описати наступним чином:

1. Час навчання: ReLU зазвичай швидша за tanh і sigmoid, бо вона простіша математично (лише max). sigmoid і tanh використовують експоненціальні обчислення, які повільніші.
2. Помилка: tanh часто дає нижчу помилку, бо вона центрована навколо нуля, що полегшує оптимізацію. Sigmoid має тенденцію до "затухання градієнта" — це коли нейрон "перестає вчитись", особливо в глибоких мережах. ReLU гарно працює в багатьох випадках, але може мати "мертві нейрони" — коли нейрон завжди дає 0.
3. Загальна стабільність: tanh — компроміс між стабільністю і продуктивністю. Вона добре працює для невеликих і середніх мереж. Sigmoid більше підходить для вихідного шару в задачах класифікації (бінарний вихід). eLU добре працює, якщо дані нормалізовані і немає великої кількості негативних значень на вході.

Висновок: На даній лабораторній роботі мною було вивчено архітектури формального нейрона, багатoshарового персептрона, а також методи їхнього навчання; ознайомитися з програмними продуктами, що моделюють персептрони.

Код програми:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import SGD
import time
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

# Генерування даних
X = np.random.rand(1000, 7)
y = np.random.rand(1000, 1)
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# Функції для одно і багатoshарової моделі

# Багатoshаровий персептрон (MLP)
def create_mlp_model(learning_rate):
    model = Sequential([
        Dense(3, input_shape=(7,), activation='tanh'),
        Dense(4, activation='tanh'),
        Dense(1, activation='tanh')
    ])
    optimizer = SGD(learning_rate=learning_rate)
    model.compile(optimizer=optimizer, loss='mse', metrics=['mae'])
    return model

# Одношаровий персептрон (SMP)
def create_single_layer_perceptron(learning_rate):
    model = Sequential([
        Dense(1, input_shape=(7,), activation='tanh')
    ])
    optimizer = SGD(learning_rate=learning_rate)
    model.compile(optimizer=optimizer, loss='mse', metrics=['mae'])
    return model

# Крок навчання
learning_rates = [0.001, 0.01, 0.05, 0.1]
mlp_results = []
smp_results = []
```

```
# Розв'язання для MLP
for lr in learning_rates:
    model = create_mlp_model(lr)

    start_time = time.time()
    model.fit(X_train, y_train, epochs=50, verbose=0)
    training_time = time.time() - start_time

    start_eval = time.time()
    train_loss, _ = model.evaluate(X_train, y_train, verbose=0)
    train_eval_time = time.time() - start_eval

    start_eval = time.time()
    test_loss, _ = model.evaluate(X_test, y_test, verbose=0)
    test_eval_time = time.time() - start_eval

    mlp_results.append({
        'Тип': 'MLP',
        'Крок навчання': lr,
        'Час навчання': training_time,
        'Час класифікації (train)': train_eval_time,
        'Помилка (train)': train_loss,
        'Час класифікації (test)': test_eval_time,
        'Помилка (test)': test_loss
    })

# Розв'язання для SMP
for lr in learning_rates:
    model = create_single_layer_perceptron(lr)

    start_time = time.time()
    model.fit(X_train, y_train, epochs=50, verbose=0)
    training_time = time.time() - start_time

    start_eval = time.time()
    train_loss, _ = model.evaluate(X_train, y_train, verbose=0)
    train_eval_time = time.time() - start_eval

    start_eval = time.time()
    test_loss, _ = model.evaluate(X_test, y_test, verbose=0)
    test_eval_time = time.time() - start_eval

    smp_results.append({
        'Тип': 'SMP',
        'Крок навчання': lr,
        'Час навчання': training_time,
        'Час класифікації (train)': train_eval_time,
        'Помилка (train)': train_loss,
        'Час класифікації (test)': test_eval_time,
        'Помилка (test)': test_loss
    })
```

```
# Виведення графіків
df_mlp = pd.DataFrame(mlp_results)
df_smp = pd.DataFrame(smp_results)

# Графік для MLP
plt.figure(figsize=(8, 5))
plt.plot(df_mlp['Крок навчання'], df_mlp['Час навчання'], marker='o')
plt.title('MLP: Залежність часу навчання від learning rate')
plt.xlabel('Крок навчання')
plt.ylabel('Час навчання (сек)')
plt.grid(True)
plt.show()

# Графік для SMP
plt.figure(figsize=(8, 5))
plt.plot(df_smp['Крок навчання'], df_smp['Час навчання'], marker='o',
color='green')
plt.title('SMP: Залежність часу навчання від learning rate')
plt.xlabel('Крок навчання')
plt.ylabel('Час навчання (сек)')
plt.grid(True)
plt.show()

# Генерування CSV файлів
df_all = pd.concat([df_mlp, df_smp])
df_all.to_csv("results.csv", index=False)
```